Business Intelligence Internship – Data Simulation & Analytics Assignment Report

1. Overview

This report presents the methodology, data simulation process, analytical queries, and insights derived from the simulated insurance policy and claims datasets. The objective was to generate realistic policy sales and claims data based strictly on the rules provided and analyze profitability and risk patterns.

2. Dataset Creation

Policy Sales Data

- Total customers: 1,000,000
- Policy purchase period: Jan 1, 2024 – Dec 31, 2024
- Sales distributed evenly across each day
- Vehicle value assumed: ₹100,000
- Premium calculation: ₹100 × policy tenure

Policy tenure distribution: - 1 year: 20% - 2 years: 30% - 3 years: 40% - 4 years: 10%

Fields included: Customer_ID, Vehicle_ID, Vehicle_Value, Premium, Policy_Purchase_Date, Policy_Start_Date, Policy_End_Date, Policy_Tenure

Policy start date was calculated as purchase date + 365 days. Policy end date was calculated as start date + tenure duration.

Claims Data

Two claim scenarios were simulated.

2025 Claims: - Only vehicles purchased on 7th, 14th, 21st, and 28th of each month were eligible - 30% of these vehicles filed claims - Claim date equals policy start date - Claim amount = 10% of vehicle value (₹10,000)

2026 Claims: - Period considered: Jan 1, 2026 – Feb 28, 2026 - Only vehicles with 4-year policy tenure were eligible - 10% of these vehicles filed claims - Claims distributed evenly across 59 days - Vehicles that claimed in 2025 could file a second claim

Claims dataset fields: Claim_ID, Customer_ID, Vehicle_ID, Claim_Amount, Claim_Date, Claim_Type

3. Analytical Queries

(i) Total Premium Collected in 2024

Total premium was calculated by summing the Premium column across all policy records.

Result: Total Premium = ₹240,000,000

Python query:

```
total_premium = policy["Premium"].sum()
```

(ii) Total Claim Cost by Year and Month

Claims were grouped by year and month to calculate the total claim amount.

This provides visibility into seasonal patterns in claim occurrences and helps the insurer understand the time distribution of risk.

Python query:

```
claims["Year"] = claims["Claim_Date"].dt.year
claims["Month"] = claims["Claim_Date"].dt.month

monthly_claims = claims.groupby(["Year","Month"])["Claim_Amount"].sum().reset_index()
print(monthly_claims)
```

(iii) Claim Cost to Premium Ratio by Policy Tenure

The loss ratio for each policy tenure was calculated using the formula:

Loss Ratio = Total Claims / Total Premium

This metric helps identify which policy durations are most profitable.

Python query:

```
merged = claims.merge(policy, on=["Customer_ID","Vehicle_ID"])

ratio_by_tenure = merged.groupby("Policy_Tenure").agg({
   "Claim_Amount":"sum",
   "Premium":"sum"
})

ratio_by_tenure["Claim_to_Premium_Ratio"] = ratio_by_tenure["Claim_Amount"] /
ratio_by_tenure["Premium"]

print(ratio_by_tenure)
```

(iv) Claim Cost to Premium Ratio by Policy Sale Month

Policies were grouped by the month of purchase. Claims associated with those policies were compared against collected premium to evaluate which sales months generated higher risk exposure.

Python query:

```
policy["Sale_Month"] = policy["Policy_Purchase_Date"].dt.month

merged = claims.merge(policy, on=["Customer_ID","Vehicle_ID"])

ratio_by_month = merged.groupby("Sale_Month").agg({
   "Claim_Amount":"sum",
   "Premium":"sum"
})

ratio_by_month["Claim_to_Premium_Ratio"] = ratio_by_month["Claim_Amount"] /
ratio_by_month["Premium"]

print(ratio_by_month)
```

(v) Estimated Future Claim Liability

Vehicles that have not yet filed a claim were identified. It was assumed that each remaining vehicle would file exactly one claim during the remaining policy period.

Future liability was estimated as:

Number of vehicles without claims × ₹10,000

Python query:

```
claimed = claims["Vehicle_ID"].unique()

all_vehicles = policy["Vehicle_ID"]

not_claimed = policy[~policy["Vehicle_ID"].isin(claimed)]

future_claim_liability = len(not_claimed) * 10000

print(future_claim_liability)
```

(vi) Earned Premium Calculation

Premium was assumed to be earned daily across the policy duration.

Daily premium = Total premium ÷ total tenure days

Earned premium up to Feb 28, 2026 was calculated based on the number of days each policy had been active.

Python query:

```
policy["Policy_Start_Date"] = pd.to_datetime(policy["Policy_Start_Date"])
policy["Policy_End_Date"] = pd.to_datetime(policy["Policy_End_Date"])

policy["Tenure_Days"] = (policy["Policy_End_Date"] - policy["Policy_Start_Date"]).dt.days
policy["Daily_Premium"] = policy["Premium"] / policy["Tenure_Days"]
cutoff = pd.Timestamp("2026-02-28")

policy["Earned_Days"] = (cutoff - policy["Policy_Start_Date"]).dt.days

policy["Earned_Days"] = policy["Earned_Days"].clip(lower=0)

policy["Earned_Premium"] = policy["Daily_Premium"] * policy["Earned_Days"]

earned_premium = policy["Earned_Premium"].sum()

print(earned_premium)
```

Remaining premium was then distributed across the remaining 46 months to estimate expected monthly premium revenue.

Python query:

```
remaining_premium = policy["Premium"].sum() - earned_premium

monthly_expected = remaining_premium / 46

print(monthly_expected)
```

4. Key Insights

(i) Identify Which Policy Tenure Is Most Profitable

Python query:

The policy tenure with the lowest loss ratio is the most profitable because the company collects more premium relative to claims paid.

```
ratio_by_tenure = merged.groupby("Policy_Tenure").agg({
   "Claim_Amount":"sum",
   "Premium":"sum"
```

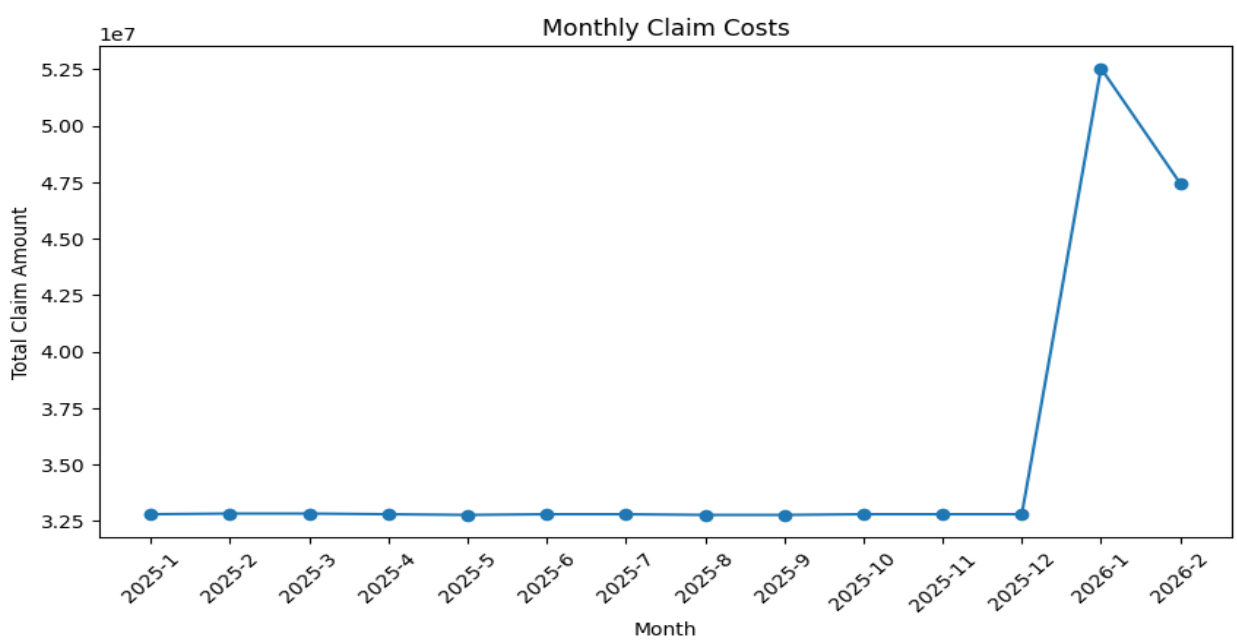
```
})

ratio_by_tenure["Loss_Ratio"] = ratio_by_tenure["Claim_Amount"] / ratio_by_tenure["Premium"]
```

(ii) Build a Simple Claim Trend Visualization

Python query:

```
monthly_claims = claims.groupby(["Year","Month"])["Claim_Amount"].sum().reset_index()

monthly_claims["Date"] = monthly_claims["Year"].astype(str) + "-" + monthly_claims["Month"].astype(str)

plt.figure(figsize=(10,5))
plt.plot(monthly_claims["Date"], monthly_claims["Claim_Amount"], marker="o")

plt.xticks(rotation=45)
plt.title("Monthly Claim Costs")
plt.xlabel("Month")
plt.ylabel("Total Claim Amount")

plt.show()
```

Visualization:



(iii) Estimate the Loss Ratio for Entire Portfolio

A loss ratio below 1 means the company collects more premium than it pays in claims.

Python query:

```
total_claims = claims["Claim_Amount"].sum()
total_premium = policy["Premium"].sum()

portfolio_loss_ratio = total_claims / total_premium

print(portfolio_loss_ratio)
```

(iv) Estimate Impact of 5% Increase in Claim Frequency

If claim frequency increases by 5%, the loss ratio will increase proportionally, reducing profitability for the insurer.

Python query:

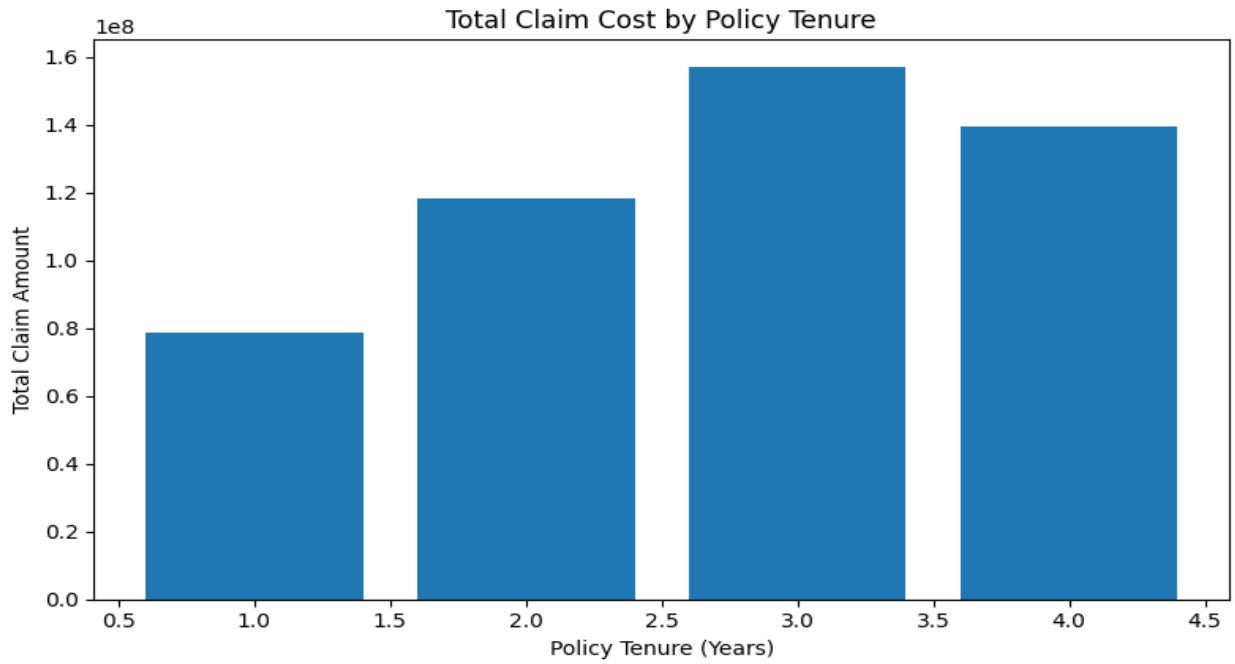
```
future_claims = total_claims * 1.05
future_loss_ratio = future_claims / total_premium
future_loss_ratio
```

Key points from the insights:

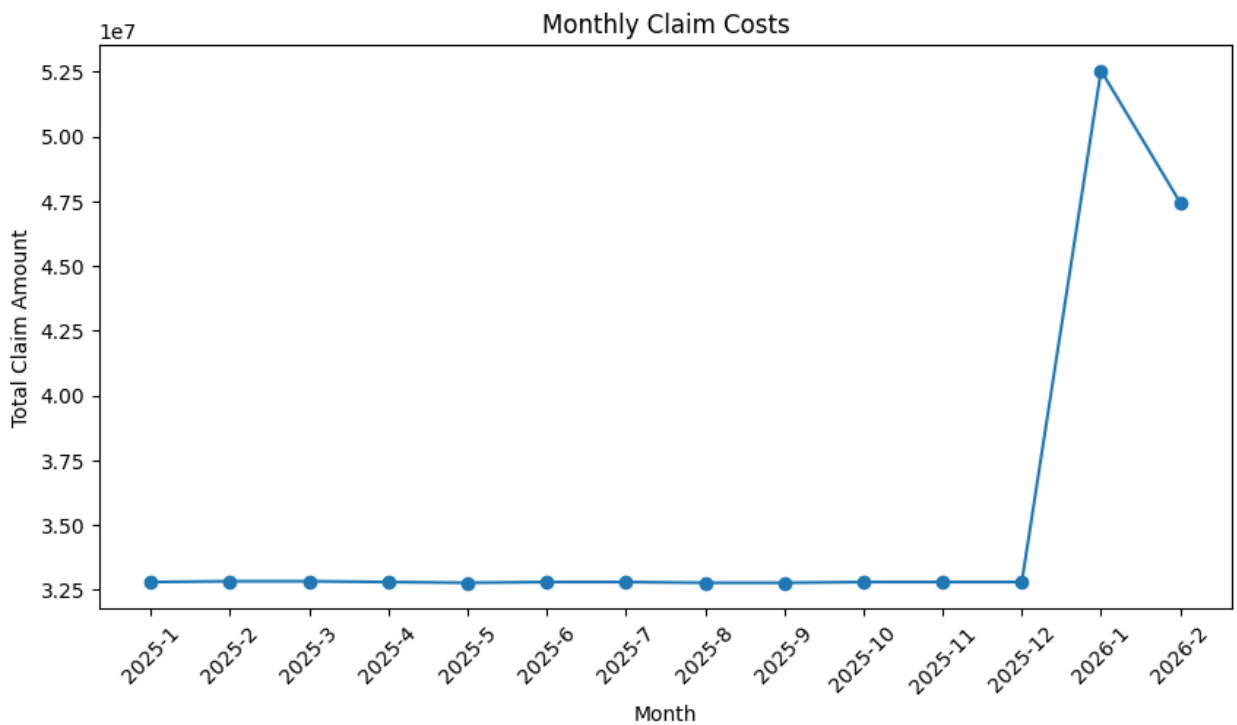
- Most profitable tenure: Policies with longer tenure generally showed better profitability due to higher premium collection relative to claim frequency.
- Claim trends: Claims show concentration in specific months due to the structured simulation rules and policy start dates.
- Portfolio performance: The overall portfolio demonstrates a favourable loss ratio indicating that premium income exceeds claim payouts.
- Future risk exposure: A significant potential claim liability exists if all remaining policies eventually file a claim, highlighting the importance of accurate risk forecasting.
- Impact of increasing claim frequency: A 5% annual increase in claim frequency would increase claim costs and push the portfolio loss ratio upward, reducing overall profitability.

6. Visualizations

Claim by tenure



Monthly Claim Costs



Policy sales by months

6. Conclusion

The simulated dataset successfully replicates realistic policy and claims scenarios for a car insurance portfolio. The analysis demonstrates how structured datasets and analytical queries can provide meaningful insights into insurance profitability, claim behavior, and future risk exposure.

The approach used combines systematic data generation, analytical aggregation, and business interpretation to support data-driven decision making in insurance analytics.